

```
In [*]: # Define a Node class for the BST
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

# Define the Binary Search Tree class
class BinarySearchTree:
    def __init__(self):
        self.root = None

    # Insert a new node in BST
    def insert(self, key):
        self.root = self._insert_recursive(self.root, key)

    def _insert_recursive(self, root, key):
        if root is None:
            return Node(key)
        if key < root.key:
            root.left = self._insert_recursive(root.left, key)
        elif key > root.key:
            root.right = self._insert_recursive(root.right, key)
        return root

    # Search for a key in BST
    def search(self, key):
        return self._search_recursive(self.root, key)

    def _search_recursive(self, root, key):
        if root is None or root.key == key:
            return root
        if key < root.key:
            return self._search_recursive(root.left, key)
        return self._search_recursive(root.right, key)

    # Delete a node from BST
    def delete(self, key):
        self.root = self._delete_recursive(self.root, key)

    def _delete_recursive(self, root, key):
        if root is None:
            return root

        if key < root.key:
            root.left = self._delete_recursive(root.left, key)
        elif key > root.key:
            root.right = self._delete_recursive(root.right, key)
        else:
            # Node with only one child or no child
            if root.left is None:
                return root.right
            elif root.right is None:
```

```
        return root.left

        # Node with two children: get inorder successor
        temp = self._minValueNode(root.right)
        root.key = temp.key
        root.right = self._delete_recursive(root.right, temp.key)

    return root

def _minValueNode(self, node):
    current = node
    while current.left is not None:
        current = current.left
    return current

# Display the BST (Inorder Traversal)
def inorder(self):
    print("Inorder Traversal:", end=" ")
    self._inorder_recursive(self.root)
    print()

def _inorder_recursive(self, root):
    if root:
        self._inorder_recursive(root.left)
        print(root.key, end=" ")
        self._inorder_recursive(root.right)

# ---- Menu-driven program ----
bst = BinarySearchTree()

while True:
    print("\n--- Binary Search Tree Menu ---")
    print("1. Insert Node")
    print("2. Delete Node")
    print("3. Search Node")
    print("4. Display (Inorder)")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ")

    if choice == "1":
        key = int(input("Enter value to insert: "))
        bst.insert(key)
        print(f"{key} inserted into BST.")

    elif choice == "2":
        key = int(input("Enter value to delete: "))
        bst.delete(key)
        print(f"{key} deleted from BST (if it existed).")

    elif choice == "3":
        key = int(input("Enter value to search: "))
        result = bst.search(key)
        if result:
            print(f"{key} found in the BST.")
```

```
        else:
            print(f"{key} not found in the BST.")

    elif choice == "4":
        bst.inorder()

    elif choice == "5":
        print("Exiting program. Goodbye!")
        break

    else:
        print("Invalid choice. Please enter a number between 1 and 5.")
```

--- Binary Search Tree Menu ---

```
1. Insert Node
2. Delete Node
3. Search Node
4. Display (Inorder)
5. Exit
Enter your choice (1-5): 1
Enter value to insert: 23
23 inserted into BST.
```

--- Binary Search Tree Menu ---

```
1. Insert Node
2. Delete Node
3. Search Node
4. Display (Inorder)
5. Exit
Enter your choice (1-5): 4
Inorder Traversal: 23
```

--- Binary Search Tree Menu ---

```
1. Insert Node
2. Delete Node
3. Search Node
4. Display (Inorder)
5. Exit
```

Enter your choice (1-5):

In []: